



Université de Haute-Alsace – ENSISA

Informatique et Réseaux – année 2025-2026

PROJET FINAL · DEEP LEARNING 1

Architectures d'apprentissage profond pour la classification de signaux ECG

Jeu de données ECG200 – UCR Time Series Archive

ÉLÈVE

Ozkosar Enes

ENCADRANT

M. Maxime Devanne

***Note sur l'utilisation d'outils d'IA :** Dans le cadre de ce projet, nous avons eu recours à un assistant IA pour deux aspects précis.*

- (1) **Architecture BiLSTM :** Après réflexion, nous avons estimé qu'un BiLSTM serait plus adapté qu'un RNN simple pour capturer les dépendances temporelles dans les deux sens. Nous avons utilisé un assistant IA pour comprendre comment implémenter cette architecture et identifier des hyperparamètres de départ raisonnables.*
- (2) **Technique ReduceLROnPlateau :** Nous avons utilisé un assistant IA pour comprendre le mécanisme de réduction adaptative du learning rate.*

Dans tous les cas, les choix finaux, l'implémentation, les expériences et l'analyse des résultats ont été réalisés intégralement par l'équipe.

Table des matières

1	Introduction	4
2	Données et prétraitement	4
2.1	Description du jeu de données ECG200	4
2.2	Normalisation Z-score	5
3	Architectures des modèles	6
3.1	MLP – Multi-Layer Perceptron	6
3.2	CNN – Convolutional Neural Network (1D)	6
3.3	BiLSTM – LSTM Bidirectionnel	7
4	Techniques de régularisation et d'optimisation	8
4.1	Dropout	8
4.2	Régularisation L2	8
4.3	Early Stopping	8
4.4	ReduceLROnPlateau (initiative personnelle)	9
4.5	Gestion du déséquilibre de classes – <code>class_weight</code> (initiative personnelle)	9
4.6	Reproductibilité	9
5	Protocole d'évaluation	9
5.1	Évaluation de la robustesse par multi-runs	9
5.2	Métriques d'évaluation	10
6	Résultats expérimentaux	10
6.1	Courbes d'apprentissage	10
6.2	Performances sur le jeu de test	11
6.3	Matrices de confusion	12
6.4	Rapports de classification détaillés	12
6.5	Étude de complexité comparative	14
7	Analyse et discussion	14
7.1	Comparaison des performances brutes	14
7.2	Analyse par classe : focus sur la détection des infarctus	14
7.3	Interprétation de la robustesse	15
7.4	Pourquoi le BiLSTM performe moins bien sur ECG200 ?	15
7.5	Recommandation pour le service de cardiologie	16
8	Conclusion	16

1 Introduction

Ce rapport présente le travail réalisé dans le cadre du projet final du cours de Deep Learning. L’objectif est de construire et comparer plusieurs architectures de réseaux de neurones profonds pour la classification automatique de signaux ECG (*électrocardiogrammes*).

Le contexte applicatif est médical : un service de cardiologie cherche à automatiser la détection de crises cardiaques (infarctus) à partir de signaux ECG. Cette détection est aujourd’hui souvent réalisée manuellement, ce qui est coûteux en temps. Un système automatique fiable permettrait de traiter un grand nombre de signaux rapidement et d’alerter le personnel médical en cas de signal anormal. Le sujet précise également que les PCs du service ne sont pas particulièrement puissants, ce qui impose une contrainte sur la complexité des modèles pouvant être déployés.

Nous travaillons sur le jeu de données **ECG200** de l’archive UCR, qui contient des séries temporelles représentant l’activité électrique cardiaque sur un battement. Chaque signal est étiqueté :

- **+1 (Normal)** : rythme cardiaque normal.
- **-1 (Infarctus)** : rythme anormal, pouvant indiquer une crise cardiaque.

Trois architectures majeures sont comparées : un **MLP** (Multi-Layer Perceptron), un **CNN** (Convolutional Neural Network, convolutions 1D), et un **BiLSTM** (LSTM bidirectionnel). Chaque modèle est évalué sur ses performances de classification, sa robustesse aux initialisations aléatoires, et sa complexité computationnelle, afin d’identifier la solution la plus adaptée à un déploiement en milieu hospitalier.

2 Données et prétraitement

2.1 Description du jeu de données ECG200

Le dataset ECG200 est un jeu de données de petite taille, ce qui en fait à la fois un avantage (temps d’entraînement court) et un défi (risque de surapprentissage, résultats moins stables statistiquement). Il est **pré-découpé** en ensemble d’apprentissage et ensemble de test par les auteurs du dataset.

Les données sont chargées directement depuis les URLs du sujet (format **.tsv**). Les labels sont convertis de $\{-1, +1\}$ vers $\{0, 1\}$ pour être compatibles avec la convention Keras pour la `binary_crossentropy`.

TABLE 1 – Répartition des classes dans ECG200

Ensemble	Total	Normal (+1)	Infarctus (-1)
Train	100	67	33
Test	100	64	36

On observe un déséquilibre de classes : les signaux normaux sont environ deux fois plus nombreux que les signaux d’infarctus. Ce déséquilibre a des conséquences pratiques importantes, discutées en section 4.5.

Chaque signal est une série temporelle de **96 points**, correspondant à un battement cardiaque complet. La figure 1 illustre la différence morphologique entre un signal normal et un signal pathologique.

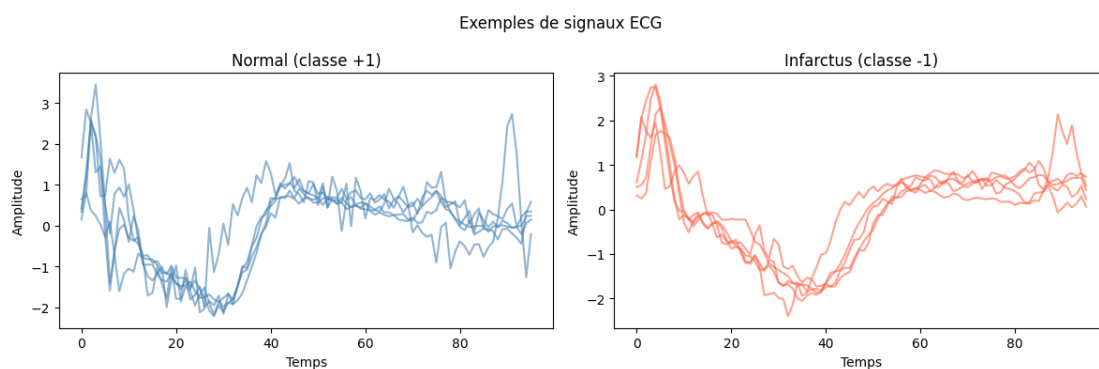


FIGURE 1 – Exemples de signaux ECG issus du jeu de données ECG200. Le signal normal est représenté en bleu, le signal d’infarctus en rouge.

2.2 Normalisation Z-score

La normalisation est indispensable en Deep Learning pour s’assurer que toutes les valeurs sont à la même échelle, ce qui facilite la convergence du gradient. Nous utilisons une **normalisation Z-score**, apprise uniquement sur les données d’entraînement. Pour chaque valeur x_i du signal :

$$x'_i = \frac{x_i - \mu_{\text{train}}}{\sigma_{\text{train}}} \quad (1)$$

où μ_{train} et σ_{train} sont calculés sur le seul ensemble d’entraînement. On applique ensuite cette transformation (sans recalculer) sur l’ensemble de test, pour éviter toute fuite d’information.

Pour les architectures CNN et BiLSTM, le signal normalisé est reformaté en séquence de forme (96, 1) pour satisfaire l’API Keras de ces couches.

3 Architectures des modèles

Pour les trois modèles, la sortie est un unique neurone avec activation sigmoïde, produisant une probabilité pour la classe « Infarctus ». La fonction de perte utilisée est la `binary_crossentropy`, adaptée à la classification binaire. L'optimiseur est **Adam** avec un taux d'apprentissage initial de 10^{-3} .

3.1 MLP – Multi-Layer Perceptron

Le MLP est l'architecture de référence. Il traite le signal ECG comme un vecteur aplati de 96 valeurs et apprend des représentations non linéaires via des couches entièrement connectées (*Dense*).

Choix de l'architecture : Deux couches cachées (64 puis 32 neurones, activation ReLU) permettent d'apprendre des représentations à plusieurs niveaux d'abstraction sans être trop lourd. Sur 100 exemples d'entraînement, un modèle plus profond risquerait de surapprendre facilement.

Nous avons ajouté de la **Batch Normalisation** après chaque couche Dense pour stabiliser l'entraînement. Cette technique normalise les activations intermédiaires à chaque mini-batch, ce qui réduit la sensibilité au taux d'apprentissage et accélère la convergence. Une **régularisation L2** ($12(1e-4)$) est également appliquée sur toutes les couches Dense.

TABLE 2 – Architecture du MLP (8 705 paramètres)

Couche (type)	Output Shape	Paramètres
Input	(None, 96)	0
Dense (64, ReLU, L2)	(None, 64)	6 208
BatchNormalization	(None, 64)	256
Dropout (0.4)	(None, 64)	0
Dense (32, ReLU, L2)	(None, 32)	2 080
BatchNormalization	(None, 32)	128
Dropout (0.3)	(None, 32)	0
Dense (1, sigmoid)	(None, 1)	33
Total entraînable		8 513

3.2 CNN – Convolutional Neural Network (1D)

Le CNN est particulièrement adapté aux séries temporelles car les convolutions 1D permettent d'extraire des motifs locaux dans le signal, quelle que soit leur position temporelle.

Pourquoi Conv1D et pas Conv2D ? Le signal ECG est unidimensionnel (un seul axe temporel). La convolution 2D est conçue pour des données avec deux dimensions spatiales comme les images. L'utiliser sur un signal 1D n'aurait pas de sens physique et nécessiterait de reshaper artificiellement les données, sans bénéfice.

Choix de l'architecture : Trois blocs Conv1D + MaxPooling1D empilés, avec des filtres croissants ($16 \rightarrow 32 \rightarrow 64$) et des noyaux décroissants ($7 \rightarrow 5 \rightarrow 3$). Cette progression permet d'apprendre d'abord des structures larges dans le signal, puis des détails fins. Le MaxPooling divise la dimension temporelle par 2 à chaque étape, réduisant le nombre de paramètres et forçant le réseau à ne garder que l'essentiel.

TABLE 3 – Architecture du CNN (33 569 paramètres)

Couche (type)	Output Shape	Paramètres
Input	(None, 96, 1)	0
Conv1D (16 filtres, k=7, L2)	(None, 96, 16)	128
MaxPooling1D (pool=2)	(None, 48, 16)	0
Conv1D (32 filtres, k=5, L2)	(None, 48, 32)	2 592
MaxPooling1D (pool=2)	(None, 24, 32)	0
Conv1D (64 filtres, k=3, L2)	(None, 24, 64)	6 208
MaxPooling1D (pool=2)	(None, 12, 64)	0
Flatten	(None, 768)	0
Dense (32, ReLU, L2)	(None, 32)	24 608
Dropout (0.5)	(None, 32)	0
Dense (1, sigmoid)	(None, 1)	33
Total		33 569

3.3 BiLSTM – LSTM Bidirectionnel

Le LSTM (*Long Short-Term Memory*) est une architecture appartenant à la famille des RNN (Réseaux de Neurones Récurrents). Contrairement à un RNN classique qui souffre du problème du gradient qui disparaît (*vanishing gradient*), le LSTM résout ce problème grâce à son mécanisme de portes (input gate, forget gate, output gate), ce qui le rend plus efficace pour capturer des dépendances temporelles.

Nous avons opté pour un **LSTM Bidirectionnel** (BiLSTM). Contrairement à un LSTM unidirectionnel qui lit le signal de gauche à droite, le BiLSTM le lit dans les deux sens simultanément, puis concatène les deux représentations.

Justification du BiLSTM : Pour une série temporelle, le contexte « futur » peut être aussi informatif que le contexte « passé ». Le BiLSTM permet de capturer ces deux directions, ce qui peut améliorer la classification. Cette architecture n'a pas été abordée

en cours. Nous avons eu recours à un assistant IA pour comprendre le principe de la bidirectionnalité et identifier des hyperparamètres de départ raisonnables.

Choix des hyperparamètres : Deux couches BiLSTM empilées (32 puis 16 unités LSTM par direction), séparées par du Dropout (0.4). La première couche retourne les séquences pour alimenter la seconde. Une régularisation L2 plus forte (1e-2) est appliquée car le BiLSTM tend davantage à surapprendre.

TABLE 4 – Architecture du BiLSTM (19 105 paramètres)

Couche (type)	Output Shape	Paramètres
Input	(None, 96, 1)	0
Bidirectional LSTM (32, seq=True, L2)	(None, 96, 64)	8 704
Dropout (0.4)	(None, 96, 64)	0
Bidirectional LSTM (16, L2)	(None, 32)	10 368
Dropout (0.4)	(None, 32)	0
Dense (1, sigmoid)	(None, 1)	33
Total		19 105

4 Techniques de régularisation et d'optimisation

4.1 Dropout

Le Dropout est une technique de régularisation vue en cours. Pendant l'entraînement, chaque neurone est désactivé aléatoirement avec une probabilité p à chaque itération. Cela empêche le réseau de trop dépendre d'un neurone particulier et l'oblige à apprendre des représentations plus robustes. Nous utilisons des taux de Dropout de 0.3 à 0.5 selon les couches et les modèles.

4.2 Régularisation L2

La régularisation L2, vue en cours, ajoute une pénalité proportionnelle au carré des poids à la fonction de perte. Cela décourage les poids trop grands et réduit le surapprentissage. Elle est appliquée sur toutes les couches convolutionnelles et denses.

4.3 Early Stopping

L'Early Stopping est vu en cours. L'entraînement est stoppé automatiquement quand la validation loss ne s'améliore plus pendant 30 époques. Nous restaurons les poids du meilleur modèle observé, et non de la dernière époque, ce qui est crucial pour ne pas récupérer un modèle qui a commencé à surapprendre.

4.4 ReduceLROnPlateau (initiative personnelle)

Cette callback réduit dynamiquement le taux d'apprentissage lorsque la validation stagne. Elle n'a pas été vue en cours. Les paramètres utilisés sont : un facteur de 0.5 (le learning rate est divisé par 2 à chaque déclenchement), une patience de 10 époques, et un plancher à 5e-6 pour éviter un learning rate effectivement nul.

La justification est simple : en fin d'entraînement, un taux d'apprentissage fixe peut être trop grand et provoquer des oscillations autour du minimum. En le réduisant progressivement, on permet une convergence plus fine. Dans les logs d'entraînement, on observe le déclenchement de cette callback plusieurs fois avant l'Early Stopping, ce qui est exactement le comportement souhaité.

4.5 Gestion du déséquilibre de classes – `class_weight` (initiative personnelle)

La classe « Normal » est deux fois plus fréquente que la classe « Infarctus ». Sans correction, le modèle tendrait à prédire plus souvent « Normal » pour maximiser l'accuracy, au détriment du rappel sur les infarctus.

Or, dans un contexte médical, un **faux négatif** (prédire « Normal » alors que c'est un infarctus) est beaucoup plus grave qu'un faux positif : un patient souffrant d'un infarctus non détecté risque sa vie. Nous avons donc utilisé le paramètre `class_weight` de Keras pour attribuer un poids inversement proportionnel à la fréquence de chaque classe :

$$w_c = \frac{N_{\text{total}}}{n_{\text{classes}} \times N_c} \quad (2)$$

Avec 67 normaux et 33 infarctus : $w_{\text{infarctus}} = 100/(2 \times 33) \approx 1.52$ et $w_{\text{normal}} = 1.0$. Chaque erreur sur un infarctus est donc pénalisée plus lourdement dans la loss.

4.6 Reproductibilité

Pour garantir la reproductibilité, les graines aléatoires sont fixées en début de script (SEED=42). Pour l'évaluation de robustesse, les graines varient intentionnellement de 0 à 4 entre les runs, ce qui permet d'observer la variabilité naturelle des modèles.

5 Protocole d'évaluation

5.1 Évaluation de la robustesse par multi-runs

Un résultat obtenu sur un seul entraînement n'est pas statistiquement fiable sur un dataset aussi petit. L'initialisation aléatoire des poids peut avoir un impact significatif

sur les résultats finaux.

Nous évaluons donc chaque modèle sur **5 runs** avec des graines différentes (0 à 4). Nous calculons la moyenne et l'écart-type de l'accuracy : $\mu \pm \sigma$. Un écart-type faible signifie que le modèle est stable et ses performances sont reproductibles, quelle que soit l'initialisation.

5.2 Métriques d'évaluation

Nous reportons les métriques suivantes :

- **Accuracy** : proportion de prédictions correctes.
- **Précision, Rappel, F1-score** par classe : métriques plus informatives en cas de déséquilibre.
- **Matrice de confusion** : visualisation des erreurs (vrais/faux positifs et négatifs).

6 Résultats expérimentaux

6.1 Courbes d'apprentissage

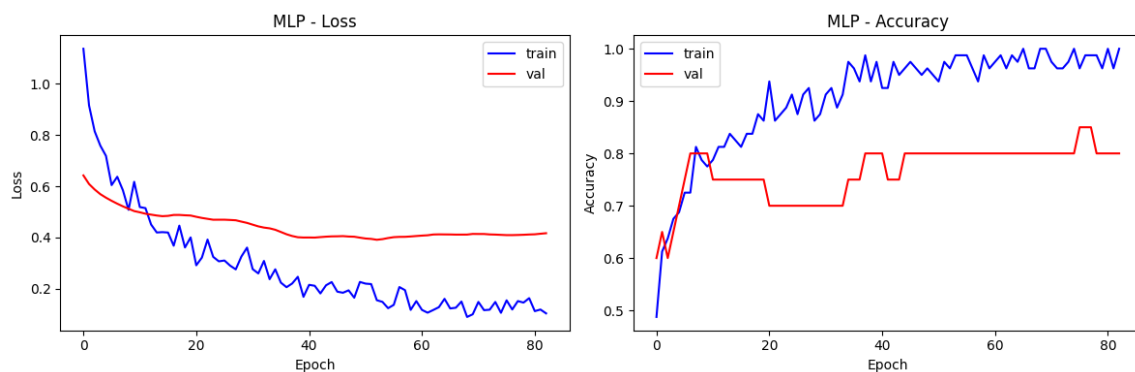


FIGURE 2 – Courbes d'apprentissage du MLP. L'Early Stopping intervient à l'époque 83 (meilleure époque : 53). `ReduceLROnPlateau` se déclenche aux époques 63, 73 et 83. La train accuracy monte jusqu'à ~ 0.96 tandis que la val accuracy se stabilise autour de 0.80, signe d'un léger surapprentissage maîtrisé.

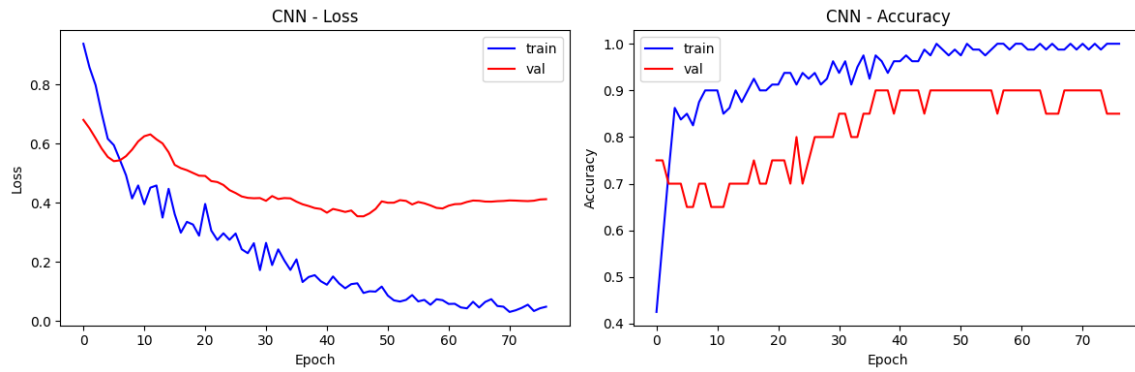


FIGURE 3 – Courbes d’apprentissage du CNN. Early Stopping à l’époque 77 (meilleure époque : 47). La train loss descend très bas (~ 0.05) tandis que la val loss stagne autour de 0.40 : le CNN surapprend davantage malgré le Dropout de 0.5, en raison de ses 33k paramètres sur 100 exemples.

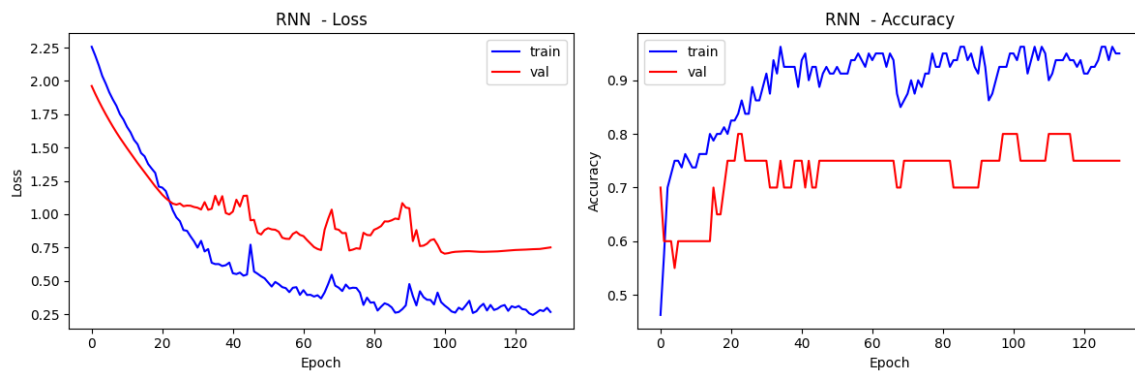


FIGURE 4 – Courbes d’apprentissage du BiLSTM. Early Stopping à l’époque 131 (meilleure époque : 101). La val accuracy oscille fortement entre 0.65 et 0.80, témoignant de la difficulté du BiLSTM à converger sur un si petit dataset.

6.2 Performances sur le jeu de test

TABLE 5 – Résultats comparatifs des trois modèles sur le jeu de test ECG200

Modèle	Acc. train	Acc. test	Robustesse (5 runs)	Params
MLP	0.96	0.87	0.8300 ± 0.0155	8 705
CNN	0.96	0.88	0.8520 ± 0.0271	33 569
BiLSTM	0.92	0.77	0.8180 ± 0.0075	19 105

6.3 Matrices de confusion

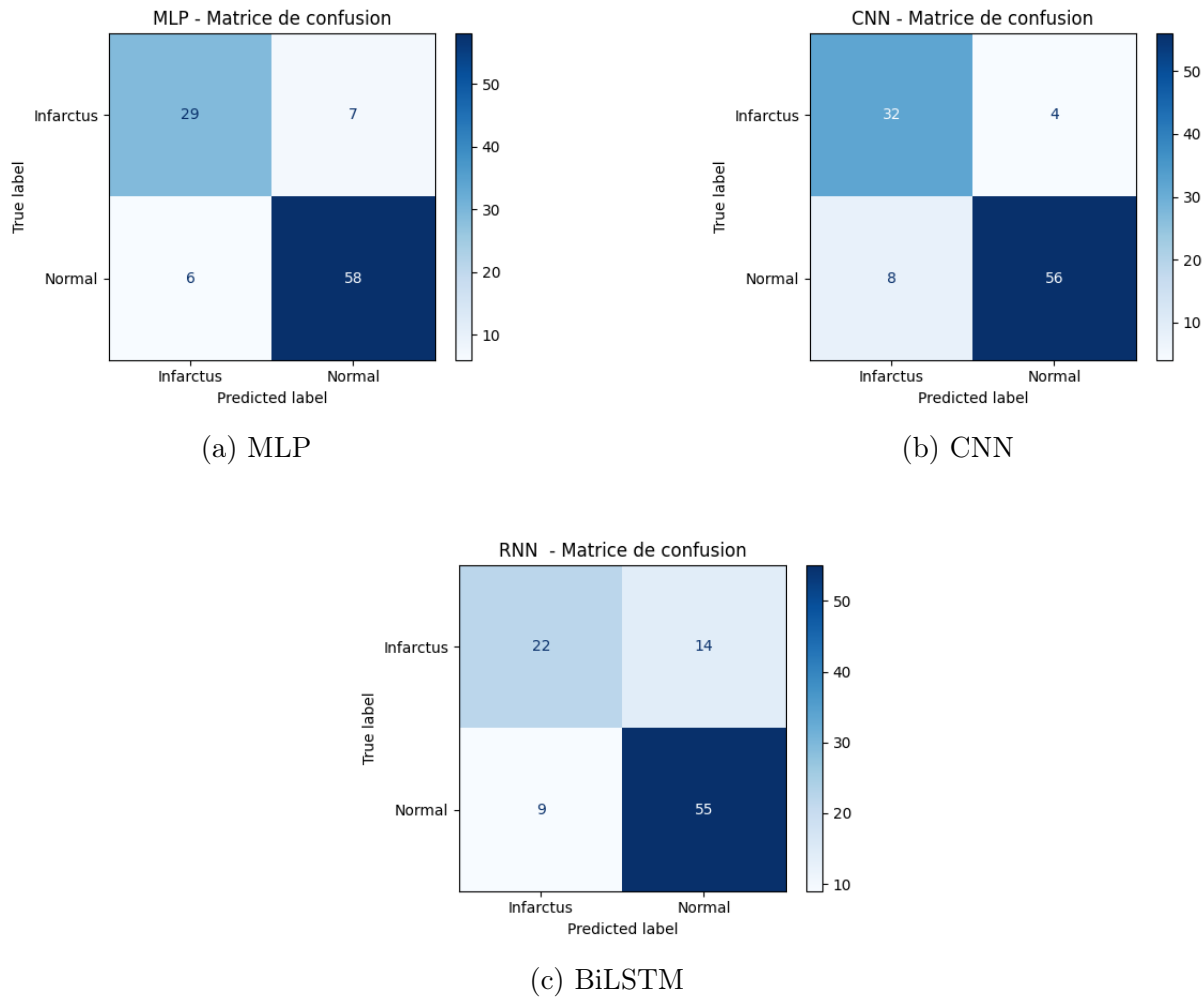


FIGURE 5 – Matrices de confusion des trois modèles.

6.4 Rapports de classification détaillés

TABLE 6 – Rapport de classification – MLP

Classe	Précision	Rappel	F1-score	Support
Infarctus (-1)	0.83	0.81	0.82	36
Normal (+1)	0.89	0.91	0.90	64
Accuracy		0.87		100
Macro avg	0.86	0.86	0.86	100
Weighted avg	0.87	0.87	0.87	100

Le MLP obtient 87% d'accuracy globale. Sur la classe Infarctus, le rappel de 0.81 signifie que 7 infarctus sur 36 ne sont pas détectés. La précision de 0.83 indique que la

majorité des alarmes déclenchées sont justifiées. C'est un bon équilibre pour un modèle aussi léger.

TABLE 7 – Rapport de classification – CNN

Classe	Précision	Rappel	F1-score	Support
Infarctus (-1)	0.80	0.89	0.84	36
Normal (+1)	0.93	0.88	0.90	64
Accuracy		0.88		100
Macro avg	0.87	0.88	0.87	100
Weighted avg	0.89	0.88	0.88	100

Le CNN obtient le meilleur rappel sur les infarctus (0.89), soit seulement 4 cas manqués sur 36. En contrepartie, sa précision de 0.80 est légèrement plus faible : il génère davantage de faux positifs (8 contre 6 pour le MLP), ce qui reste cliniquement acceptable puisqu'un faux positif mène simplement à un examen supplémentaire.

TABLE 8 – Rapport de classification – BiLSTM

Classe	Précision	Rappel	F1-score	Support
Infarctus (-1)	0.71	0.61	0.66	36
Normal (+1)	0.80	0.86	0.83	64
Accuracy		0.77		100
Macro avg	0.75	0.74	0.74	100
Weighted avg	0.77	0.77	0.77	100

Le BiLSTM affiche les résultats les plus faibles : un rappel de 0.61 sur les infarctus signifie que 14 cas sur 36 ne sont pas détectés, soit près de 40%. Dans un contexte médical, ce taux de faux négatifs est problématique. Le F1-score de 0.66 sur la classe Infarctus confirme que ce modèle est le moins adapté au déploiement en l'état.

6.5 Étude de complexité comparative

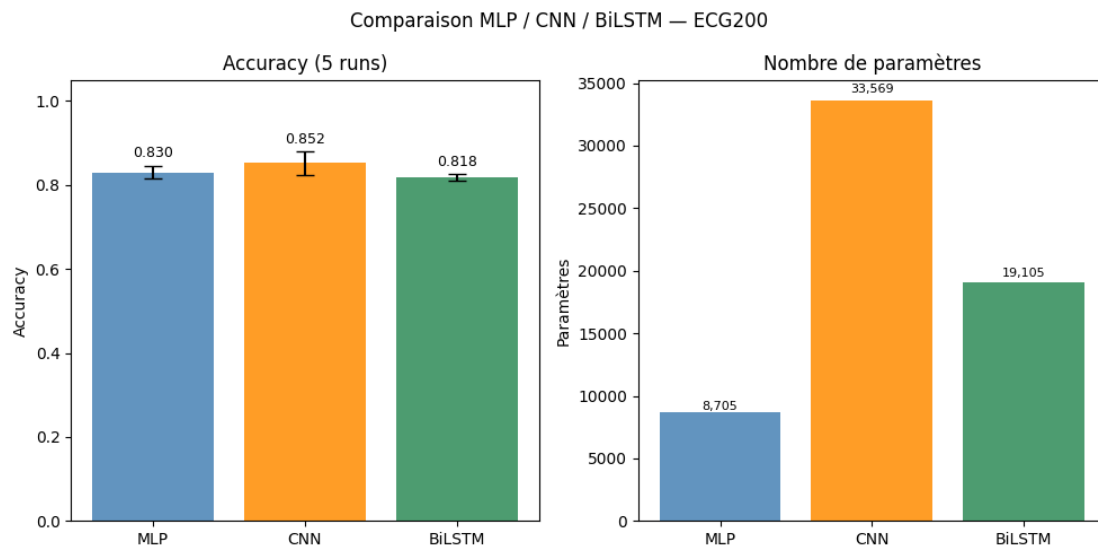


FIGURE 6 – Comparaison MLP / CNN / BiLSTM sur ECG200. À gauche : accuracy moyenne sur 5 runs avec barres d’erreur $\pm\sigma$. À droite : nombre de paramètres par modèle.

TABLE 9 – Comparaison de la complexité des modèles

Modèle	Params	Taille (KB)	Époques	Meilleure ép.
MLP	8 705	~34	83	53
CNN	33 569	~131	77	47
BiLSTM	19 105	~75	131	101

7 Analyse et discussion

7.1 Comparaison des performances brutes

En regardant uniquement l’accuracy sur un run unique, le CNN obtient 88%, suivi du MLP à 87%, et du BiLSTM à 77%. Cependant, l’accuracy seule n’est pas suffisante, surtout en présence d’un déséquilibre de classes. Un modèle prédisant systématiquement « Normal » obtiendrait déjà 64% sans avoir rien appris d’utile.

7.2 Analyse par classe : focus sur la détection des infarctus

Dans ce contexte médical, la classe « Infarctus » est la plus critique. On cherche un **rappel élevé** pour cette classe : le modèle doit rater le moins possible d’infarctus réels. Un faux négatif (infarctus non détecté) peut avoir des conséquences graves pour le patient, tandis qu’un faux positif provoque simplement un examen médical supplémentaire.

- Le **CNN** obtient un rappel de **0.89** sur les infarctus : seulement **4 infarctus manqués** sur 36. C'est excellent. En contrepartie, 8 faux positifs, ce qui reste cliniquement acceptable.
- Le **MLP** a un rappel de **0.81** (7 infarctus manqués) avec une précision de 0.83 et 6 faux positifs. Bon compromis global.
- Le **BiLSTM** n'atteint qu'un rappel de **0.61** (**14 infarctus manqués** sur 36). C'est problématique : près de 40% des infarctus ne seraient pas détectés.

7.3 Interprétation de la robustesse

Les résultats de robustesse sur 5 runs amènent des conclusions plus nuancées :

- Le **BiLSTM** présente le **plus faible écart-type** ($\sigma = 0.0075$). Malgré ses moins bonnes performances brutes, il est le plus stable et prévisible d'un run à l'autre.
- Le **MLP** atteint $\mu = 0.830$ avec $\sigma = 0.0155$: bon compromis entre performance et stabilité.
- Le **CNN** obtient la meilleure accuracy moyenne ($\mu = 0.852$) mais avec l'écart-type le plus élevé ($\sigma = 0.0271$). Ses performances varient davantage selon l'initialisation, ce qui reflète sa tendance à surapprendre (33k paramètres pour 100 exemples).

Du point de vue de l'efficacité, le MLP est remarquable : il atteint 83% de robustesse avec seulement 8 705 paramètres, soit presque 4 fois moins que le CNN pour une performance similaire.

7.4 Pourquoi le BiLSTM performe moins bien sur ECG200 ?

Plusieurs facteurs expliquent ces résultats inférieurs :

(1) **Manque de données** : Les LSTM ont besoin de beaucoup de données pour apprendre des dépendances temporelles stables. Avec 100 exemples d'entraînement, le modèle n'a pas assez de matière pour généraliser.

(2) **Longueur des séquences** : 96 points est relativement court. Les avantages du LSTM – prévus pour des dépendances à très long terme – sont donc peu exploités ici.

(3) **Oscillations de la validation** : Les courbes d'apprentissage montrent une val accuracy très irrégulière, témoignant de difficultés de convergence malgré la régularisation L2 forte et le Dropout.

(4) **Durée d'entraînement** : Le BiLSTM nécessite 131 époques (contre 77–83 pour les autres) avant l'Early Stopping, ce qui implique un temps d'entraînement plus important.

Malgré ces limites sur ECG200, le BiLSTM reste une architecture pertinente : si le service de cardiologie collectait davantage de signaux, il serait probablement la meilleure option à terme.

7.5 Recommandation pour le service de cardiologie

Le sujet précise que les machines du service ne sont pas très puissantes. Cette contrainte est déterminante pour le choix du modèle à déployer.

En termes de performances pures, le **CNN** est le meilleur modèle (rappel Infarctus de 0.89, accuracy moyenne de 85.2%). Mais il nécessite 33 569 paramètres, soit ~ 131 KB, et présente la plus grande variabilité (± 0.027).

En tenant compte des contraintes matérielles et de fiabilité, le **MLP apparaît comme le meilleur choix pour un déploiement réel** :

- **Léger** : 8 705 paramètres (~ 34 KB), soit environ 4 fois moins que le CNN. Il peut fonctionner sans GPU sur une machine modeste, voire être intégré dans un système embarqué.
- **Performant** : 87% d'accuracy, rappel Infarctus de 0.81, très proches du CNN.
- **Stable** : $\sigma = 0.0155$, des performances reproductibles entre déploiements.
- **Rapide à l'inférence** : architecture simple, adaptée à un traitement en temps quasi-réel sur machine modeste.
- **Simple à maintenir** : architecture claire, facile à auditer et à mettre à jour.

En synthèse : si les ressources matérielles le permettent et si maximiser le rappel sur les infarctus est la priorité absolue, le CNN est à privilégier. Dans le cas général du service de cardiologie tel que décrit dans le sujet, le MLP est la solution la plus pragmatique et la plus adaptée au déploiement en conditions réelles.

8 Conclusion

Ce projet nous a permis de comparer trois architectures de Deep Learning – MLP, CNN 1D et BiLSTM – sur la classification de signaux ECG (dataset ECG200). Les principaux enseignements sont :

1. Le **CNN** est l'architecture la plus performante en termes d'accuracy brute et de rappel sur la classe Infarctus (88%, rappel 0.89). Ses filtres convolutionnels capturent efficacement les patterns locaux du signal ECG, ce qui explique ses bonnes performances sur ECG200.
2. Le **MLP** offre le meilleur compromis performance/complexité pour un déploiement hospitalier : 87% d'accuracy avec 4 fois moins de paramètres que le CNN. C'est la solution recommandée pour les ressources limitées.
3. Le **BiLSTM** est le plus stable ($\sigma = 0.0075$) mais moins performant sur ce petit dataset. Ses avantages théoriques se manifesteraient pleinement avec davantage de données.

4. **L'évaluation sur 5 runs** est indispensable : les résultats d'un run unique peuvent être trompeurs. Le BiLSTM affiche 77% en run unique mais 81.8% de robustesse moyenne.
5. Les initiatives personnelles – `class_weight`, `ReduceLROnPlateau` et BiLSTM bidirectionnel – ont contribué à améliorer les résultats, notamment sur la détection des infarctus.